

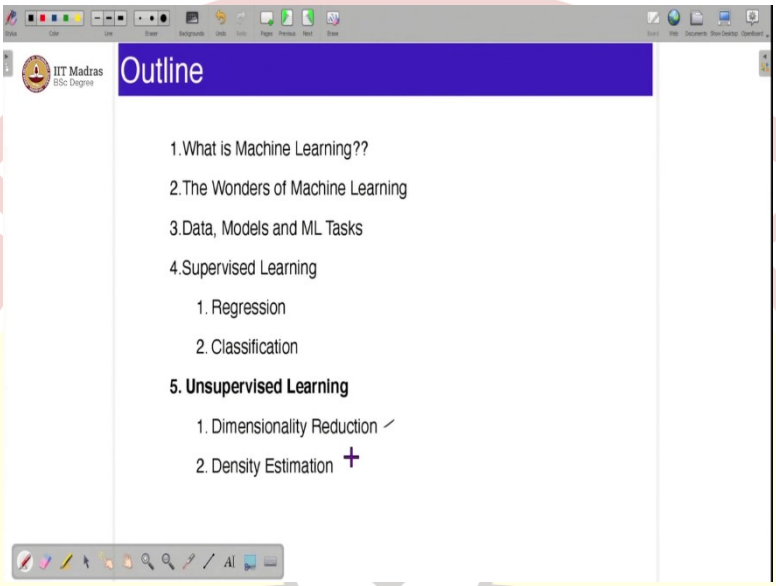
IIT Madras

ONLINE DEGREE

Machine Learning Foundations
Professor Harish Guruprasad Ramaswamy
Department of Computer Science and Engineering
Indian Institute of Technology, Madras
Unsupervised Learning Dimensionality Reduction

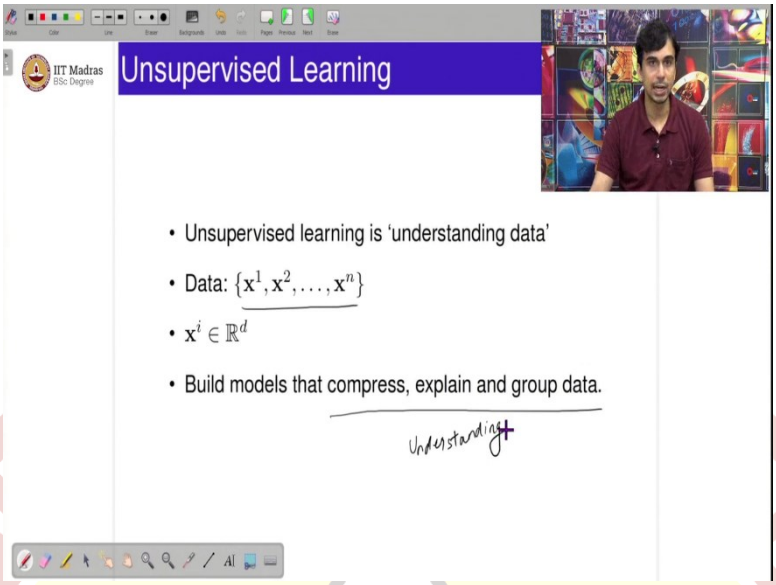
Hello, everyone, and welcome to another lecture on machine learning foundations.

(Refer Slide Time: 0:23)



In the previous lectures, we introduced the supervised learning paradigm and went into detail on the two main problems, two main types of supervised learning tasks which are regression and classification. In this lecture, we are going to start on unsupervised, on the unsupervised learning paradigm, and the two main tasks associated with it, which are dimensionality reduction and density estimation.

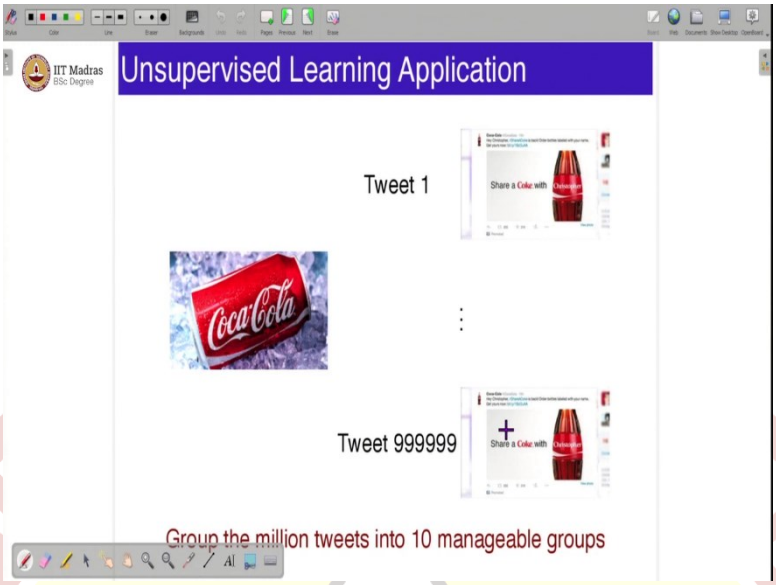
(Refer Slide Time: 0:47)



In contrast to supervised learning, which had very clear and marked goals and ways of quantifying them, unsupervised learning is typically much more vague. And unsupervised learning typically is used as a pre-processing step, and not as an end in itself. Vaguely, unsupervised learning can be viewed as base for understanding data. And data in our context here is simply going to be a collection of vectors.

Note that in contrast to supervised learning, which always had two pairs of xi yi, here you have just xi, which are just a collection of d dimensional vectors. The goal of unsupervised learning is to build models that compress, explain and group data, which is what I am broadly grouping as understanding. We will explain what all of these mean with two specific examples, which are dimensionality reduction, and density estimation.

(Refer Slide Time: 1:53)



Here is an example of how unsupervised learning can be useful. Unsupervised learning, as I mentioned, is typically not used as an end in itself, because the outputs of unsupervised learning algorithms by themselves are not useful. But after human interpretation, and after other machine learning tasks, they can become very valuable.

For example, let us say you are a marketing manager at Coca Cola. And your job is to collect the tweets about Coca Cola and summarize them to your boss. So, that is your job. And let us say in any given week, there are let us say 1 million, 10 million say, a million tweets about Coca Cola that happen in a given week. There is no way you can show all the million tweets to your boss and explain what each tweet is, it is just not possible.

A reasonable thing to do would be if you can group these, let us say a million tweets into 10 distinct groups. So, maybe you can have one group of people who are all just taking selfies with coke in a new place. Or maybe there is another group of tweets, which are from other brands, which has, which are doing some co-branding or we can have another group of feeds, which are all about people who are promoting Coke, and paid by Coke.

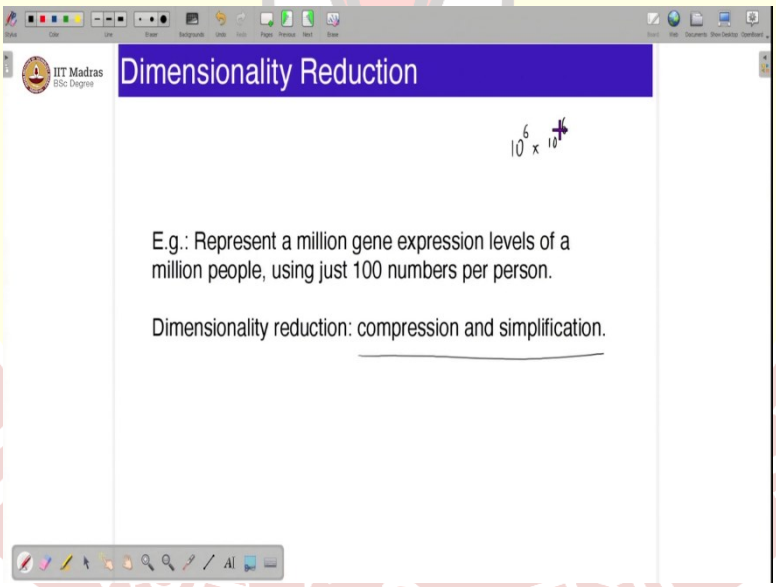
So, there are several groups which can potentially think of, and if you can, even without actually giving this information, if you could potentially group these million tweets into 10, manageable groups, and understand what these groups are by going through these groups. And then you can go to your manager and say, well, there are 10 types of tweets that happened this week.

And the type 1 tweets are called tweets by people who are buying coke for the first time and tweeting about it from a store near their house. And group 2 is all people who are all businesses which are collaborating with coke and so on. And you can easily summarize such a situation to your boss and the boss, you can do that in a reasonable amount of time, and you can get your job done.

So, this is an example of unsupervised and you can note that the what the unsupervised learning does is simply group that tweets into 10 groups. So, that is the, that is what it does anything beyond that interpreting these groups is typically the job of the human being, because groups by themselves are meaningless. Only when you assign coherent meaning to such a group, would, it, would it be actionable or useful.

So, that is done by the human which is in this case would be you. That is the reason why is it unsupervised learning is typically not an end in itself, but rather a pre-processing stage which is used by other processes.

(Refer Slide Time: 4:53)



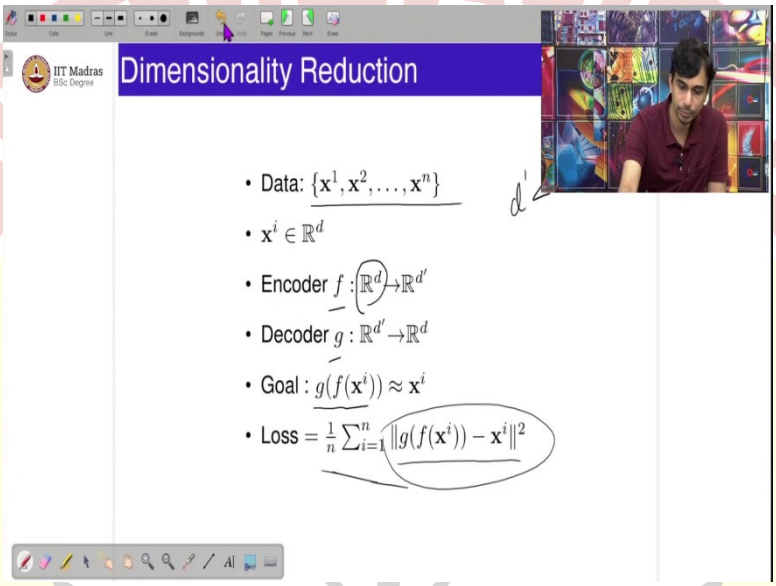
Here is one example of one concrete example of unsupervised learning which is dimensional reduction. So, the goal of dimensionality reduction is compression and simplification. Where would it be useful for? For example, let us say you have your genetic company and you want to export or comprehensively store the million gene expression levels of a million people.

So, you have you have gene results of a million people. And each, each person has, let us say, a million genes. So, what have is you have to we have computed the gene expression levels of these million genes, a million people. So, in principle, it is a $10^6 \times 10^6$ matrix. It will have

million people each having million genes. So, if you have the entire data, that is, that is a 10 power 6 + 10 power 6 numbers, and there is no way you can, let us say, transmit this data from one lab to another lab, it is just not possible.

What would be nice would be if you can compress this data into a simpler format, which can be used for transmitting. So, that is one reasonable goal to have. And dimensionality reduction is one of the main tools that you can use for such a task for such a goal.

(Refer Slide Time: 6:17)



Formally writing mathematic, mathematical terms, that is you might have data in D dimensional vectors x1 to xn. And the goal of dimensionality reduction is to come up with two models, unlike all the other previous cases like classification or regression, where the goal of the dimension, the learning algorithm was to come up with a single model, the goal of a dimensionality reduction algorithm is to come up with two models, an encoder and a decoder.

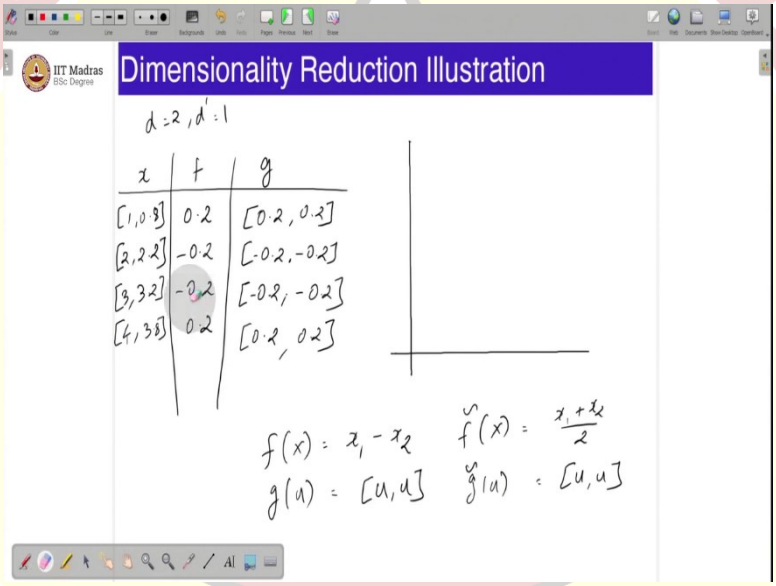
What does the encoder do? The encoder is a function which takes in a D dimensional vector and outputs a d' dimensional vector. Typically, the' is less than D, typically d' is much lesser than d. Effectively, the encoder compresses a D dimensional vector into a d' dimensional vector. And the decoder essentially hopes to undo the effect of the encoder, the decoder takes a d' dimensional vector and outputs a D dimensional vector.

What is the goal of the encoder and the decoder so that if you take any input in your data, and you encode it and you decode it again, you should get back the original data. The goal is that your g of f of xi should be equal to xi, but you are willing to relax it and say that, as long as it

is approximately equal to xi, you are happy. So, that is the goal now. The goal is g of f of xi is approximately equal to xi. And how do you measure approximation?

Well, a reasonable way of measuring approximation would be to simply measure take g of f of xi - xi and view that as a vector and compute the norm square or the length squared of that vector. I, if this was equal to 0, then the g of f, norm of g of f of xi - xi the whole square if that was 0, clearly, g f of xi =xi, but in general it need not be equal to 0, you are looking for a pair of encoder and decoder so that g of f of xi - xi, the norm squared of that is as small as possible. And once again, you are averaging this over all n possible input, all the n inputs as 1 by n, sigma i equal to 1 to n, norm of g of f of xi - xi the whole square.

(Refer Slide Time: 8:33)



Here, is a very simple illustration to show how the dimensionality reduction can work. So, let us take a very simple example here. Let us say you have d =2, and d' =1 in this case, it is a very simple example. You have 4, let us say how 4 training points, this is the 4 training points, this is your x, the 4 training points are let us say, 1 , 0.8. The second training point is let us say 2 , 2.2. Third is 3 , 3.2. And the fourth is 4 , 3.8.

This is your 2 dimensional data and you are looking for an encoder, which takes in a 2 dimensional vector and outputs a scalar. And a decoder which takes in a scalar or a one-dimensional vector and outputs a two-dimensional vector. We will consider 2 possibilities for this. So, let us take one, let us say f of x =one possibility is x1 - x2 and g of u is u , u.

Note that f is a mapping from R2 to R, which means that x is a 2 dimensional vector but x1 - x2 is clearly a scalar, x1 - x2 is clearly a scalar. f is a mapping which takes in a 2 dimensional

vector and outputs a scalar and g is a mapping which takes a decoder which takes in the scalar u and outputs a vector 2 dimensional vector u , u.

So, now, let us what f is, so f would be this case would be 0.2, - 0.2, - 0.2 and 0.2. So, why is that, because x1 - x2 is 1 - 0.8 is 0.2, 2 - 2.2 is - 0.2 and so on. And what is g, g would essentially take in the output of f, the encode, the f of x and g of f of x should ideally be equal to x for the training points x1, x2, x3 and x4.

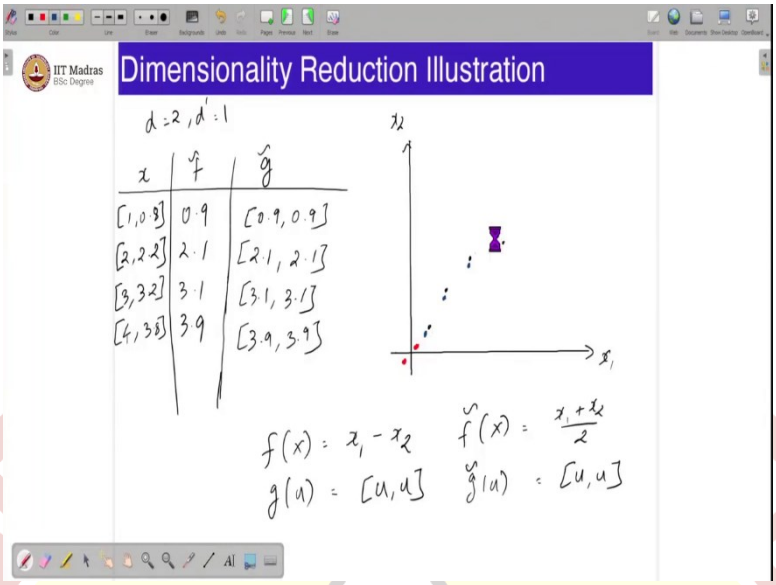
If you do that, for this the first training point g of 0.2 is the vector 0.2 , 0.2. You can immediately see that 0.2 , 0.2 is not equal to 1 , 0.8 and it is quite distance away. We can do the same thing for this the second data point will be - 0.2 , - 0.2 and the third - 0.2 , - 0.2 and 0.2 , 0.2 you have encoded this using the encoder and decoder given you have encoded the input and decoded it and this is what you get.

You can clearly see this is not a very good encoder and decoder because both 1 , 0.8 and 4 , 3.8 if you run it through the encoder and decode it again, you get the same 0.2 , 0.2 even though you have two very distinct inputs getting mapped to the same output. So, this is not doing a very good job of compressing losslessly.

So, it is doing some compression you are representing a two dimensional vector by a one dimensional scalar but, you are losing out, or I mean you are the goal is to get even though you lose, even though you store using lesser number of space or lesser number of numbers, but you want to get back the original input. You should, your goal is to not lose over the input, but on the other hand, but this particular example of encoder and decoder is not doing a good job of doing that.

Now, let us consider another encoder decoder path. Let us have f tilde and g tilde. f tilda of x is let us say, x1 + x2 by 2 and g tilde of u is u , u. If you do the same thing for f tilde and g tilde, let us see what that gives us. I will just erase this.

(Refer Slide Time: 13:23)



Tilda and g tilde, f tilde for the first data point will be x1 + x2 by 2 which is 0.9. 2 + 2.2 by 2 is 2.1. 3 + 3.2 by 2 is 3.1 and 4 + 3.8 by 2 is 3.9. This is the encoder output and the decoder if you apply it to the output of the encoder, it would be 0.9 , 0.9, 2.1 , 2.1. 3.1 , 3.1 and 3.9 , 3.9.

You can clearly see that this is a much better encoder decoder than the previous one. Why is that because 1 , 0.8 and 0.9 , 0.9 is pretty close and 4 , 3.8 and 3.9 , 3.9 is also pretty close and all the 2 , 2.2 and 2.1 , 2.1 is close. 3 , 3.1, 3.1 , 3.1 is also close.

Let us actually do a plotting for this. So, this is the x1 axis. This is the x2 axis you had 4 points which correspond to say 1 , 0.8, 2 , 2.2, 3 , 3.2 and 4 , 3.8. So, you had these 4 data points. If you had used the encoder and decoder given by f and g, the 4, f of g of x1, g of f of x1 would be, 0.2 , 0.2, g of f of x2 would be - 0.2 , - 0.2, and g of f of x3 would also be - 0.2 , 0.2, and g of f of x4 would also be equal to 0.2 , 0.2.

You would have the 4 outputs of the encoder decoder pair would essentially be these four numbers 0.2 , 0.2 and - 0.2 , - 0.2, which is clearly not a very good approximation. But on the other hand, if you do the same thing for f tilde and g tilda, you get really good numbers. So, you get for the first f tilde g tilde applied on x1, you would get 0.9 , 0.9, which is close to here.

And 2.1 , 2. 1, which is close here, I am not being very exact here to just get you the approximate idea of where the four points are. You can see that f of, f tilde of g, g tilde of f tilde of xi is very, very close to xi when compared to g of f of xi. So, we will say that g tilde , f tilde form a much form a better encoder decoder, or f tilde , g tilde form a better encoder decoder than f , g.

Once again, this is very simplified dimensionality reduction algorithm where we are already giving two pairs of encoder decoder f , g or f tilde , g tilde and asking the dimensionality reduction algorithm to choose among these two. In the reality, it would choose the best function from Rd to Rd' as for encoder and the best function Rd' to Rd for decoder, infinite number of functions.

That is what real dimensionality reduction learning algorithm would do. Here is a tie illustration where it just chooses between these two different encoder decoder pairs. So, with that, I think we can have the dimensionality reduction part of the thing.

